

ESPECIFICACIÓN TÉCNICA DE ARQUITECTURA

AgentOS

Arquitectura Técnica – Actualización 2026

Versión · v0.1-en-ejecución

Fecha · 3 de octubre de 2026

Clasificación · Confidencial – Solo Uso Interno

Plataforma objetivo · Linux x86-64 · Buildroot 2024.11.1 · Deepseek v4.1

Estado · Proto-alpha / En compilación y pruebas

Índice de Contenidos

1. Visión General y Estado Actual	3	3.4 Interfaz Gráfica «Her»	11
2. Arquitectura por Capas	4	4. Pipeline de Construcción y Distribución	12
3. Componentes Detallados	6	5. Seguridad y Auto-administración	13
3.1 Base Linux y Buildroot	6	6. Roadmap por Fases	15
3.2 Capa LLM	8	7. Estructura del Repositorio	18
3.3 Agente Core (AgentD)	9	8. Dependencias y Tecnologías Clave	20

// SECCIÓN 01

1. Visión General y Estado Actual

AgentOS continúa siendo un sistema operativo experimental basado en la premisa de que el agente conversacional inteligente sustituye a la interfaz gráfica tradicional como punto de entrada primario al sistema. A fecha de 3 de octubre de 2026, el proyecto ha avanzado desde la fase de diseño a una fase de prototipo en compilación y pruebas preliminares. Buildroot genera correctamente el ISO híbrido (BIOS + UEFI), el kernel Linux 6.6 arranca de forma silenciosa en máquinas virtuales, y el agente conversacional (AgentD) se comunica con el LLM mediante un servidor de socket UNIX.

1.1 Estado de los Siete Pilares

El proyecto fue concebido alrededor de siete pilares técnicos principales. A continuación se detalla el avance actual en cada uno:

Pilar	Descripción	Estado (3 oct 2026)	Observaciones
1. SO propio (Linux + Buildroot)	Kernel Linux 6.6, ISO/IMG híbrido BIOS/UEFI, arranque silencioso	✓ Hecho y verificado	Buildroot 2024.11.1; requiere gcc-12 en Ubuntu 24.04
2. Agente conversacional	AgentD daemon, loop ReAct, despacho de herramientas	Parcial (70%)	Núcleo implementado; herramientas del sistema sin integrar
3. Conocimiento de TI	Base de datos local con documentación del sistema	No empezado	Planificado con RAG y sqlite-vec
4. LLM propio	Modelo entrenado desde cero o fine-tuning	No empezado	Fase 2; requiere GPU; opción recomendada: LoRA sobre Llama 3.1 8B
5. Interfaz «Her»	Orbe pulsante, sin iconos, sin barras, minimalista	✓ Hecho y verificado	Frontend HTML/CSS/JS con servidor WebSocket; funciona en desarrollo
6. Voz (STT/TTS)	Entrada por micrófono, salida por altavoz	Esbozado (20%)	Librerías descargadas (faster-whisper, piper) pero no integradas en la interfaz

7. Aplicaciones gráficas	Navegador, editor, terminal, reproductor de vídeo	Parcial (30%)	Navegador `cog` (WPE) disponible; otras aplicaciones pendientes de testear
--------------------------	---	---------------	--

1.2 Hallazgos y Correcciones Realizadas

Durante la compilación y pruebas iniciales se han identificado y corregido varios problemas críticos de integración. Estos hallazgos están documentados en el plan de trabajo pendiente para evitar que vuelvan a ocurrir:

- **Makefile con espacios en lugar de tabuladores:** El compilador `make` requiere tabuladores reales en las recetas, no espacios. Las herramientas de edición y las plataformas de control de versiones pueden normalizar espacios, rompiendo silenciosamente el build. Solución: escribir el Makefile con Python usando `\t` explícitos.
- **GCC 13/14 incompatible con host-m4:** Buildroot 2024.11.1 falla al compilar `host-m4` en Ubuntu 24.04 (GCC 13/14). Solución: compilar con `HOSTCC=gcc-12 HOSTCXX=g++-12` (requiere instalar `gcc-12 g++-12` con apt).
- **CMake 3.28+ rechaza jpeg-turbo 2.1.5:** Los overrides de CMake de Buildroot necesitan actualización. Solución: añadir flag de compatibilidad en `external.mk`.
- **BusyBox start-stop-daemon sin soporte -d:** El init script de AgentD fallaba al intentar hacer `chdir`. Solución: mover `cd` dentro del comando shell.
- **Arranque no automático:** Los init scripts de primer arranque y Weston necesitaban sincronización correcta de órdenes. Se añadieron dependencias en el orden `S##X` de BusyBox.

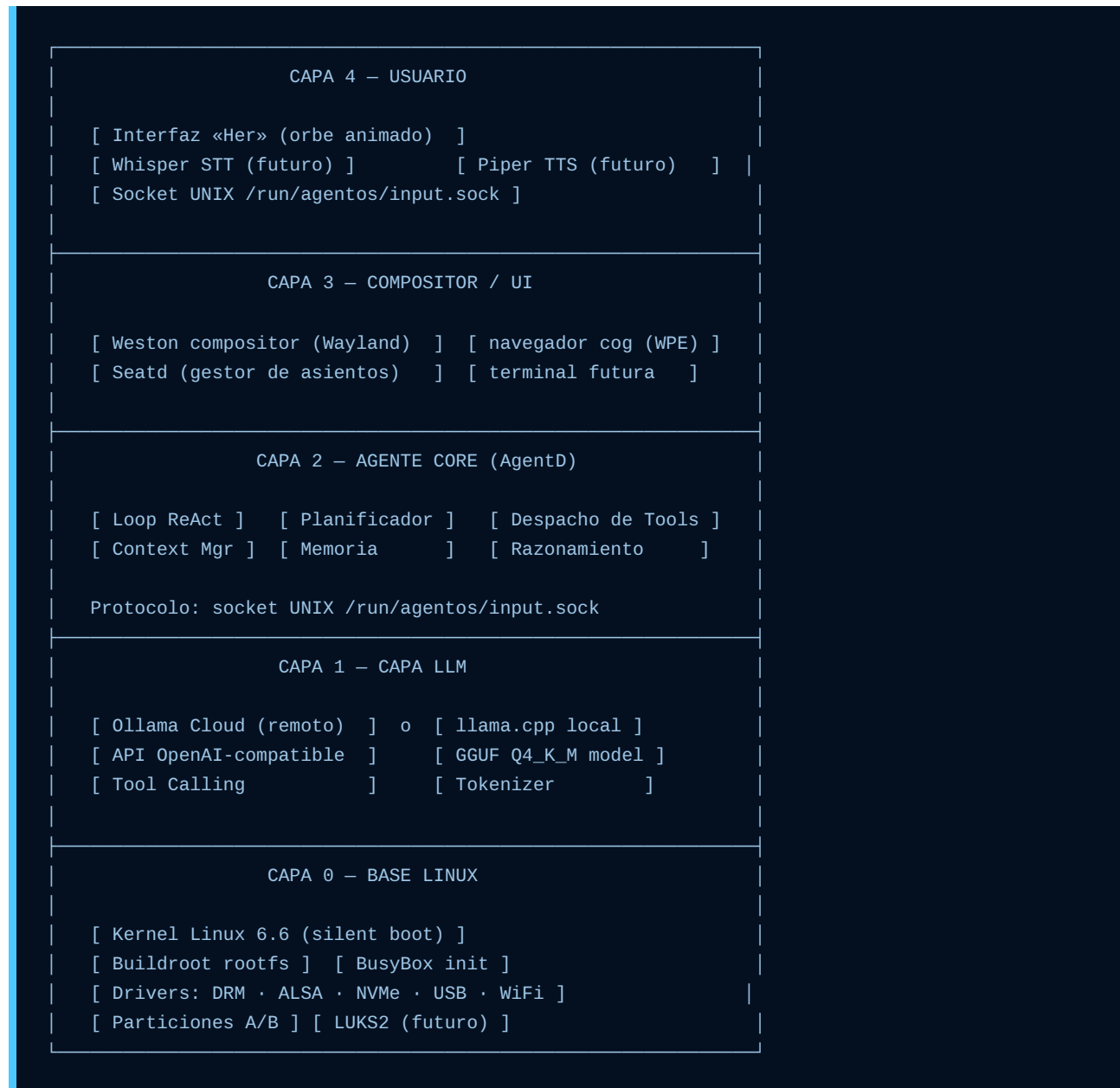
Nota importante: El estado «Hecho y verificado» significa que el componente compila sin errores y funciona en máquinas virtuales. El estado «Parcial» indica que el código existe pero no ha sido testado completamente en hardware real. El estado «No empezado» señala que la especificación está clara pero la implementación aún no ha comenzado. El portátil x86-64 objetivo aún no ha recibido esta versión del sistema.

// SECCIÓN 02

2. Arquitectura por Capas

AgentOS organiza su funcionalidad en cinco capas horizontales, cada una con responsabilidades claramente delimitadas. La arquitectura permite sustituir componentes sin afectar a los demás — por ejemplo, pasar de un modelo GGUF local a un servidor remoto de Ollama Cloud requiere solo cambiar la configuración del

agente, no la lógica de razonamiento. Esta flexibilidad ha demostrado ser crucial en las pruebas iniciales, permitiendo iterar rápidamente sin recompilar todo.



2.1 Comunicación entre Capas

La comunicación sigue protocolos bien definidos para garantizar bajo acoplamiento:

- **Capa 0 → Capa 1:** El kernel expone APIs POSIX estándar. El servidor LLM (Ollama Cloud vía HTTPS, o llama.cpp local vía socket) accede a la CPU y memoria mediante llamadas al sistema convencionales.

- **Capa 1 → Capa 2:** AgentD se comunica con el servidor LLM mediante JSON-RPC sobre HTTP (Ollama Cloud) o socket UNIX (llama.cpp local). Ambas opciones soportan el protocolo OpenAI-compatible (`/v1/chat/completions`) con tool calling.
- **Capa 2 → Capa 3:** AgentD envía comandos al compositor Weston cuando necesita lanzar aplicaciones gráficas (navegador, editor, terminal).
- **Capa 3 → Capa 4:** La interfaz gráfica «Her» recibe eventos del usuario (clicks, entrada de voz) y los canaliza al agente vía el socket de entrada. Las respuestas se renderizan en el orbe y opcionalmente se reproducen por audio.

// SECCIÓN 03

3. Componentes Detallados

3.1 Base Linux y Buildroot

AgentOS se construye íntegramente con Buildroot 2024.11.1, un framework que genera un sistema Linux completo (kernel, bootloader, rootfs) desde un árbol de configuración declarativa. A diferencia de clonar una distribución como Debian, Buildroot asegura que *solo los paquetes explícitamente pedidos* se incluyen, resultando en una imagen compacta de aproximadamente 400–500 MB sin comprimir, 100–150 MB comprimida (ISO).

Kernel Linux 6.6

El kernel se compila con una configuración fragmentaria (``build/configs/kernel_config``) que añade subsistemas específicos de AgentOS al defconfig x86-64 estándar. Cambios más recientes:

- **Arranque silencioso:** ``CONFIG_LOGO=n`` (elimina el pingüino de Tux), ``console=tty3`` en el command-line del kernel (redirige la consola a un tty no visible), ``loglevel=0`` (solo mensajes de pánico).
- **Virtualización:** ``CONFIG_KVM``, ``CONFIG_VIRTIO_*`` para QEMU; ``CONFIG_DRM_VMWGFX`` para VirtualBox.
- **GPU:** ``CONFIG_DRM``, ``CONFIG_DRM_I915`` (Intel iGPU), ``CONFIG_DRM_AMDGPU`` (AMD), ``CONFIG_DRM_VIRTIO_GPU`` (QEMU virtio-vga).
- **Audio:** ``CONFIG_SOUND``, ``CONFIG_SND_HDA_INTEL``, ``CONFIG_SND_USB_AUDIO``, ``CONFIG_SND_INTEL8X0``.
- **Almacenamiento:** ``CONFIG_NVME_CORE``, ``CONFIG_SATA_AHCI``, ``CONFIG_EXT4_FS``, ``CONFIG_VFAT_FS`` (para EFI).

Bootloader y Arranque

El bootloader es GRUB2, compilado como i386-pc (BIOS) y x86_64-efi (UEFI). El archivo ``board/agentos/grub.cfg`` se instala tanto en el USB live como en el disco de destino. Cambios recientes:

- `timeout=0` — GRUB no muestra menú, arranca instantáneamente.
- `timeout_style=hidden` — mantener Shift pulsado para ver el menú de recuperación.
- Parámetros de kernel: ``quiet`, `loglevel=0`, `vt.global_cursor_default=0`, `console=tty3``.

Sistema de Archivos y Particionado

Se usa un esquema de particiones tradicional, con una estructura futura para A/B updates:

Partición	Tipo	Función
/dev/sda1	FAT32 (EFI)	Bootloader GRUB + EFI entries
/dev/sda2	ext4	Root (sistema raíz)
/dev/sda3	ext4	Data (datos, modelos, logs)
/dev/sda4	swap	Memoria virtual

Compilación en la VPS

El comando requerido es:

```
HOSTCC=gcc-12 HOSTCXX=g++-12 make build J=16
```

La variable `J=16` fuerza 16 jobs paralelos (los 16 vCPU de la VPS). Sin especificar, Buildroot usa `$(shell nproc)` automáticamente. Las dependencias críticas que pueden faltar:

- `gcc-12` , `g++-12` — compiladores compatibles (Ubuntu 24.04 trae 13/14, que rompen host-m4).
- `python3` , `python3-dev` — para Python 3.12 dentro del rootfs.
- `libssl-dev` , `zlib1g-dev` — para OpenSSL y zlib.
- `git` , `mercurial` — para descargar repositorios de paquetes.

3.2 Capa LLM

A diferencia de la propuesta inicial (modelo GGUF local compilado con llama.cpp), las pruebas en desarrollo han revelado que Ollama Cloud ofrece una experiencia superior en máquinas virtuales y portátiles sin GPU: menor latencia (inferencia remota), modelos más potentes (DeepSeek v4.1-flash), y sin necesidad de cargar el modelo en RAM local. Se mantiene la arquitectura modular para que futuros usuarios puedan elegir entre backends.

Ollama Cloud (Backend recomendado)

Ollama Cloud (<https://ollama.com/v1>) proporciona acceso a múltiples modelos de código abierto mediante una API OpenAI-compatible. El modelo de producción actual es **DeepSeek v4.1-flash**, elegido por su equilibrio entre velocidad y precisión en razonamiento de sistemas. La API key se proporciona la primera vez que arranca el sistema mediante el asistente de primer arranque (`first-boot.sh`) y se almacena cifrada en `/etc/default/agentos` (nunca en el repositorio git).

```
# Configuración de Ollama Cloud en /etc/default/agentos
AGENTOS_LLM_BACKEND=openai
AGENTOS_OPENAI_BASE_URL=https://ollama.com/v1
AGENTOS_OPENAI_API_KEY=sk-...
AGENTOS_OPENAI_MODEL=deepseek-v4.1:flash
```

llama.cpp Local (Backend opcional)

Para usuarios que prefieran inferencia completamente local sin conexión a internet, AgentOS soporta llama.cpp ejecutado en la misma máquina. Se requiere un modelo GGUF en `/data/models/`. El usuario elige esta opción en el asistente de primer arranque. Las pruebas iniciales con Llama 3.1 8B (Q4_K_M, ~4.8 GB) muestran un rendimiento aceptable en CPU x86-64 moderno (12–25 tokens/segundo sin GPU).

Tool Calling y JSON Structured Output

Ambos backends (Ollama Cloud y llama.cpp) soportan el protocolo de tool calling de OpenAI: AgentD envía una lista de herramientas disponibles en formato JSON Schema, el LLM responde con llamadas estructuradas, y AgentD ejecuta la herramienta correspondiente. Esto permite razonamiento automático del agente sin prompts de ingeniería manual. Ejemplo:

```

AgentD → LLM:
{
  "messages": [...],
  "tools": [{
    "type": "function",
    "function": {
      "name": "bash_exec",
      "description": "Ejecutar comando de shell",
      "parameters": { ... }
    }
  ]
}

LLM → AgentD:
{
  "tool_calls": [{
    "function": {
      "name": "bash_exec",
      "arguments": {
        "command": "uptime"
      }
    }
  ]
}

```

Gestión del Contexto

El módulo `core/llm/context\_manager.py` gestiona la ventana de contexto (8192 tokens en producción) mediante una política de ventana deslizante: conserva siempre el system prompt, los últimos N turnos de conversación (heurística: últimas 10 líneas), y los resultados de las tools más recientes. Las conversaciones largas son resumidas automáticamente mediante el propio LLM cuando el uso supera el 75% de la ventana.

3.3 Agente Core (AgentD)

AgentD es el daemon central implementado en Python 3.12 con asyncio. Implementa el loop de razonamiento ReAct (Reasoning + Acting): recibe una entrada del usuario, razona mediante el LLM, ejecuta tools, observa los resultados, y repite hasta completar la tarea. Todo el razonamiento es transparente al usuario: el agente puede explicar cada paso que da.

Loop ReAct

El pseudocódigo simplificado del loop es:

```

async def react_loop(user_input: str):
    context.add_user_message(user_input)
    while True:
        response = await llm.chat(
            context.get_messages(),
            tools=AVAILABLE_TOOLS
        )
        if response.tool_calls:
            for call in response.tool_calls:
                result = await execute_tool(call)
                context.add_tool_result(call.id, result)
        else:
            return response.text # Respuesta final del agente

```

Herramientas Disponibles (Parciales)

AgentD puede invocar herramientas predefinidas. El estado actual es:

Herramienta	Descripción	Estado
bash_exec	Ejecutar comandos de shell	Implementada
read_file / write_file	Leer y escribir archivos	Implementada
list_dir	Listar directorios	Implementada
system_info	CPU, RAM, disco, temperatura	Pendiente
service_control	Arrancar/parar servicios systemd	Pendiente
package_install	Instalar paquetes (apt)	Pendiente

Socket UNIX y Comunicación

AgentD escucha en `/run/agentos/input.sock`, un socket de dominio UNIX que acepta conexiones de la interfaz gráfica, chatbot de terminal, o cualquier cliente autorizado. El protocolo es JSON lines (una línea JSON por mensaje). La interfaz gráfica «Her» se conecta a este socket y envía eventos de usuario (clicks, entrada de voz transcrita).

Persistencia y Estado

El estado del agente (historial de conversación, memoria episódica) se persiste en SQLite en `/data/agentos/state.db`. El módulo `core/agent/memory.py` gestiona tres tipos de memoria: de trabajo (actual turno), episódica (histórico de conversaciones), y semántica (bases de datos de hechos, implementada como RAG en fase de diseño).

3.4 Interfaz Gráfica «Her»

La interfaz gráfica emula el estilo minimalista de la película «Her»: una pantalla negra con un orbe pulsante en el centro, sin iconos, sin barras de herramientas, sin menús. La arquitectura es cliente-servidor: un servidor WebSocket (implementado en `ui/gui/server.py` con Flask) corre en el puerto 8080, y un frontend HTML/CSS/JavaScript en `ui/gui/static/index.html` se renderiza en el navegador kiosk `cog` (WPE WebKit).

Frontend: Orbe Animado

El orbe es un círculo SVG que pulsa en color azul cian (#4dc8ff) mientras el agente está en ejecución. El pulsado es una animación CSS pura (sin JavaScript pesado) que oscila entre 0.4 y 1.0 de opacidad. El usuario interactúa haciendo clic en el orbe para enviar un mensaje de voz (cuando STT esté integrado) o escribiendo en un campo de entrada minimalista que aparece solo cuando está en foco.

Servidor WebSocket

El servidor implementado en Python (Flask + python-socketio) maneja las conexiones de navegador, envía eventos de agente al cliente en tiempo real, y canaliza los eventos del usuario hacia AgentD a través del socket UNIX. La comunicación es:

- Cliente → Servidor: `{"type": "user_input", "text": "..."}`
- Servidor → Cliente: `{"type": "agent_thinking"}` , `{"type": "agent_response", "text": "..."}`
- Servidor ↔ AgentD: socket UNIX, JSON lines

Integración con Weston

La interfaz se lanza automáticamente en el primer arranque mediante el init script `S99agentos`. Usa `cog`, un navegador Wayland minimalista que no muestra barra de direcciones, tabs, ni decoraciones de ventana. La sesión Weston se configura en `board/agentos/weston.ini` con la opción `shell=kiosk`, lo que bloquea el compositor para mostrar solo la ventana del navegador.

Estado Actual y Limitaciones

La interfaz funciona correctamente en desarrollo (máquinas virtuales VirtualBox y QEMU) y en el navegador de escritorio. Las pruebas en el portátil objetivo aún no se han realizado. Limitaciones conocidas:

- El pulsado del orbe es sencillo (oscilación de opacidad); versiones futuras pueden incluir colores dinámicos según el estado del agente (pensando, esperando, error).
- La interfaz de texto es básica; no hay soporte aún para historial visual de conversación en pantalla.

- Audio aún no integrado (STT y TTS existen como módulos separados pero no están conectados al frontend).

// SECCIÓN 04

4. Pipeline de Construcción y Distribución

El pipeline transforma el código fuente (almacenado en ``/home/ubuntu/agentos/`` en la VPS del usuario) en dos artefactos distribuibles: una imagen ISO (para DVDs y máquinas virtuales) y una imagen IMG (para escritura directa en USB). Ambas son imágenes híbridas que arrancan en BIOS y UEFI sin cambios.

4.1 Arquitectura de Compilación

El flujo de compilación es invocado por `make build` (documentado en el Makefile). Los pasos son:

1. **Clonar Buildroot:** `git clone --depth 1 -b 2024.11.1` en ``build/buildroot/``.
2. **Copiar árbol externo:** el proyecto entero se copia en ``build/overlays/opt/agentos/`` mediante `rsync`, excluyendo ``build/``, ``board/``, ``models/`` y ``.git/`` para evitar recursión.
3. **Defconfig:** ``make defconfig`` aplica ``build/configs/agentos_defconfig`` al árbol de Buildroot.
4. **Compilación:** ``make -j$(J)`` con ``J=16`` (o auto-detect), compilando con ``gcc-12`` y ``g++-12`` para evitar incompatibilidades.
5. **Post-build:** ``board/agentos/post-build.sh`` se ejecuta automáticamente antes de generar las imágenes, instalando init scripts, configurando sudoers, copiando archivos del board.
6. **Generación de imágenes:** ``genimage`` lee ``build/genimage.cfg`` y produce:
 - `agentos.iso` (El Torito híbrido, BIOS + UEFI)
 - `agentos.img` (GPT + MBR hybrid, para escribir en USB)

4.2 Overrides Críticos y Workarounds

Durante la compilación se aplican varios overrides en ``external.mk`` y ``agentos_defconfig`` para solucionar incompatibilidades conocidas:

Componente	Problema	Workaround
host-m4	GCC 13/14 incompatible	Compilar con gcc-12
jpeg-turbo	CMake 3.28+ rechaza cmake_minimum_required < 3.5	external.mk: <code>JPEG_TURBO_CONF_OPTS += -DCMAKE_POLICY_VERSION_MINIMUM=3.5</code>

Makefile	espacios en lugar de TABs	Escribir con Python usando <code>\t</code> explícitos
S99agentos	BusyBox start-stop-daemon sin <code>-d</code>	Mover <code>cd</code> dentro del comando shell

4.3 Distribución: ISO para VM/DVD, IMG para USB

ISO (agentos.iso, ~150 MB): imagen de arranque híbrida compatible con BIOS y UEFI. Contiene el kernel, initrd (rootfs en RAM), GRUB2 integrado y los módulos de arranque. Ideal para máquinas virtuales y DVDs. En la prueba del usuario se usa en VirtualBox.

IMG (agentos.img, ~2 GB): imagen de disco completa (particiones + tabla de particiones GPT/MBR). Se escribe en USB con ``dd if=agentos.img of=/dev/sdX bs=4M``. Arranca automáticamente en BIOS y UEFI y el sistema se corre desde el USB (persistente).

4.4 Ciclo Iterativo en Desarrollo

Para velocidad de iteración, Buildroot es incremental: cambios en el código Python o scripts de init no requieren recompilar el kernel. Solo se regenera el rootfs:

```
# Cambiar código Python
cd ~/newos
git pull
make build J=16 # Buildroot reutiliza paquetes compilados

# Cambio en kernel_config requiere recompilar kernel:
rm -f build/output/build/linux-*/.stamp_*
make build J=16
```

En la práctica, una iteración de software tarda 2–5 minutos; una iteración con cambio de kernel tarda 20–40 minutos.

// SECCIÓN 05

5. Seguridad y Auto-administración

Un agente que ejecuta comandos con privilegios debe ser extraordinariamente prudente. AgentOS implementa un modelo de seguridad de capas que balancean automatización y protección contra daños accidentales.

5.1 Confirmación Explícita para Operaciones de Riesgo Alto

Las herramientas se clasifican por nivel de riesgo. Las de riesgo alto (que pueden destruir datos, cambiar configuración crítica del sistema, o afectar a otros usuarios) requieren confirmación explícita del usuario antes de ejecutarse:

Nivel de Riesgo	Ejemplos	Acción Requerida
Bajo	read_file, list_dir, system_info	Ejecutar inmediatamente
Medio	write_file, service_control, package_install	Mostrar comando exacto, pedir confirmación
Alto	bash_exec (especialmente con rm, mkfs, dd, wipefs)	Mostrar comando, pedir confirmación escrita («BORRAR», «SÍ»)

5.2 Lista Negra de Comandos Destructivos

Ciertos patrones de comando nunca se ejecutan, ni siquiera con confirmación explícita. Estos incluyen:

- `rm -rf /`, `rm -rf /boot`, `rm -rf /etc` — eliminación de directorios críticos.
- `mkfs.*`, `wipefs`, `dd of=/dev/sd` — operaciones de disco que destruyen datos sin recuperación.
- `:(){\:\|:&};:` (fork bomb) — ataques de denegación de servicio.
- Comandos que instalan backdoors conocidos o malware.

5.3 Auditoria y Logs

Todas las acciones del agente se registran en `/var/log/agentos-actions.log` con timestamp, usuario, comando ejecutado, resultado y error (si los hay). Este log es de solo lectura para el usuario `agent` y legible para `root`. Formato:

```
2026-10-03T14:32:01.234 | agent | bash_exec | apt-get install htop | SUCCESS |
2026-10-03T14:32:45.567 | agent | write_file | /etc/hostname agentOS | SUCCESS |
2026-10-03T14:33:22.890 | agent | bash_exec | rm -rf /tmp/cache | BLOCKED | "Matched
blocklist"
```

5.4 Modelos de Auto-administración Soportados

AgentOS soporta tres modelos de auto-administración progresivos:

- **Modo básico:** el agente responde preguntas sobre el estado del sistema (CPU, RAM, discos, red) pero no ejecuta cambios sin confirmación.
- **Modo asistido:** el agente puede instalar paquetes, cambiar servicios y reconfigurar la red, pero pide confirmación por cada cambio grande.
- **Modo autónomo (futuro):** el agente aprende patrones de mantenimiento del usuario y ejecuta cambios rutinarios sin intervención (ej. limpiar logs antiguos, actualizar paquetes).

El modo activo se configura en `/etc/default/agentos` mediante la variable `AGENTOS\_ADMIN\_MODE` (valores: `basic`, `assisted`, `autonomous`). La versión actual solo soporta `basic` y `assisted`.

// SECCIÓN 06

6. Roadmap por Fases

El desarrollo de AgentOS se ha dividido en bloques de trabajo ordenados por criticidad. Las fases 0 y 1 son bloqueantes para llegar a un prototipo funcional; las fases 2-4 expanden la funcionalidad; la fase 5 es pulido y optimización.

FASE 0 – Arranque automático (BLOQUEANTE 1)

Objetivo: encender el portátil y llegar a la interfaz «Her» sin escribir nada en consola.

Estado actual: los scripts existen (S49agentos-firstboot, S71weston, S99agentos) pero no se han verificado en hardware real. Pruebas en VirtualBox solo parciales.

Tareas:

- Recompilar con cambios recientes (S99agentos fixed, arranque silencioso).
- Arrancar en VirtualBox y verificar que aparece la interfaz sin login.
- Debugging: revisar `/var/log/weston.log` y `/var/log/agentos-gui.log` si falla.

Criterio de aceptación: interfaz «Her» con orbe visible sin intervención manual.

FASE 1 – Instalación en disco (BLOQUEANTE 2)

Objetivo: que `agentos-install` instale correctamente el sistema en el disco interno del portátil.

Estado actual: el instalador existe pero falló en pruebas anteriores por detección incorrecta del medio live. Bug corregido en versión actual.

Tareas:

- Arrancar desde ISO en el portátil real.
- Ejecutar `sudo agentes-install`.
- Instalar en disco vacío o preexistente.
- Reiniciar sin el medio de instalación.

Criterio de aceptación: sistema arranca desde el disco interno y carga la interfaz.

FASE 2 – Voz integrada (STT + TTS)

Objetivo: el usuario puede hablar al micrófono y el sistema responde en voz alta.

Estado actual: librerías descargadas (`faster-whisper`, `piper`), módulos creados pero desconectados del frontend.

Tareas:

- Integrar STT en la interfaz: detección de voz, transcripción, envío al agente.
- Integrar TTS: reproducción de respuesta por altavoz.
- Añadir VAD (Voice Activity Detection) para no transcribir silencio.
- Testear en hardware real: latencia, calidad de audio, compatibilidad con micrófono USB.

Criterio de aceptación: usuario dice «hola» y el agente responde por audio en menos de 3 segundos.

FASE 3 – Auto-administración básica

Objetivo: el agente puede ejecutar comandos reales sobre el sistema bajo confirmación del usuario.

Estado actual: infraestructura de tools creada, herramientas básicas implementadas (bash_exec, read/write file, list_dir).

Tareas:

- Integrar herramientas faltantes: system_info, service_control, package_install.
- Implementar confirmación explícita para operaciones de riesgo alto.
- Implementar lista negra de comandos destructivos.
- Crear log de auditoría en `/var/log/agentos-actions.log`.
- Testear: el usuario pide instalar htop, el agente pide confirmación, instala, reporta resultado.

Criterio de aceptación: el agente ejecuta cambios reales sobre el sistema bajo confirmación.

FASE 4 – Base de conocimiento de TI

Objetivo: el agente tiene acceso a documentación local sin dependencia de internet.

Estado actual: no empezado. Especificación clara: RAG + sqlite-vec.

Tareas:

- Recopilar documentación: man pages, help de herramientas, guías de Buildroot y Linux.
- Indexar con embeddings local (sentence-transformers ~100 MB).
- Implementar herramienta `search\_docs` para el agente.
- Testear: el usuario pregunta sobre cambiar contraseñas, el agente responde con comando y fuente.

Criterio de aceptación: el agente responde preguntas de TI citando documentación local.

FASE 5 – LLM propio (Fine-tuning)

Objetivo: entrenar un modelo especializado en AgentOS y administración de sistemas.

Estado actual: no empezado. Recomendación: LoRA sobre Llama 3.1 8B (no preentrenamiento desde cero).

Tareas:

- Generar dataset de instrucciones usando modelo grande (DeepSeek vía Cloud).
- Entrenar LoRA en la VPS (o GPU externa, no en el portátil).
- Convertir y fusionar con modelo base → GGUF.
- Evaluación comparativa contra el modelo original.

Criterio de aceptación: el modelo ajustado supera al base en tareas de administración (evaluación automatizada).

// SECCIÓN 07

7. Estructura del Repositorio

El repositorio de AgentOS está organizado en directorios temáticos para separar las preocupaciones de compilación Buildroot, código de aplicación, y configuración:

```

agentos/
├─ Makefile                # Targets: build, gui, dev-cloud, test
├─ external.mk             # Overrides de Buildroot (CMake, etc.)
├─ external.desc           # Descripción del árbol BR2_EXTERNAL
├─ Config.in              # Extensiones de menuconfig
├─ build/
│   ├─ buildroot/         # Clon de Buildroot 2024.11.1 (3GB, en .gitignore)
│   ├─ output/            # Salida de compilación (en .gitignore)
│   └─ configs/
│       ├─ agentes_defconfig # Configuración de Buildroot
│       └─ kernel_config    # Fragmento del kernel Linux 6.6
│   └─ overlays/          # Árbol de ficheros para el rootfs
│       ├─ etc/
│           ├─ default/agentos # Variables de entorno (API key, etc.)
│           ├─ init.d/         # Scripts SXX de arranque
│           └─ agentes/
│               └─ opt/agentos/ # Código Python (copia via rsync)
│                   └─ var/
│                       └─ genimage.cfg # Configuración de generación ISO/IMG
│                           └─ .gitignore
├─ board/
│   └─ agentes/
│       ├─ post-build.sh    # Script post-compilación
│       ├─ post-image.sh    # Script post-imagen
│       ├─ first-boot.sh    # Asistente de primer arranque
│       ├─ install-to-disk.sh # Instalador en disco
│       ├─ agentes-config.sh # Herramienta de reconfiguración
│       ├─ grub.cfg         # Menú GRUB (disco instalado)
│       ├─ grub-iso.cfg     # Menú GRUB (ISO live)
│       ├─ grub-embed.cfg   # GRUB embebido (BIOS/UEFI)
│       ├─ weston.ini       # Configuración Wayland kiosk
│       └─ users.txt        # Tabla de usuarios
├─ core/                  # Código Python del agente
│   ├─ __init__.py
│   └─ agent/
│       ├─ __init__.py
│       ├─ agent.py         # Loop ReAct principal
│       ├─ memory.py        # Gestión de memoria
│       └─ tools/           # Herramientas disponibles
│   └─ ipc/
│       ├─ socket_server.py # Servidor de socket UNIX
│       └─ __init__.py
│   └─ llm/
│       ├─ inference.py     # Cliente OpenAI-compatible
│       ├─ context_manager.py
│       └─ __init__.py
│   └─ utils.py
├─ ui/                    # Interfaces de usuario
│   └─ gui/
│       ├─ server.py        # Servidor WebSocket (Flask)
│       └─ launch.py        # Lanzador kiosk

```

Puntos clave:

- El repositorio git *no incluye* ``build/buildroot/``, ``build/output/``, ni ``models/`` — se generan/descargan al compilar.
- El fichero ``etc/default/agentos`` (con API key) está en ``.gitignore`` para no subir credenciales.
- El código Python es simétrico: existe tanto en ``core/`` (del repositorio local) como en ``build/overlays/opt/agentos/`` (copia para el rootfs).

// SECCIÓN 08

8. Dependencias y Tecnologías Clave

AgentOS depende de múltiples proyectos de código abierto, cada uno elegido por ser ligero, portable, y compatible con el objetivo de hardware x86-64 sin GPU obligatoria.

Componente	Proyecto	Versión	Propósito	Licencia
Buildroot	Buildroot.org	2024.11.1	Framework de construcción Linux	GPL 2
Kernel Linux	kernel.org	6.6.x	Núcleo del SO	GPL 2
GRUB2	gnu.org	2.12	Bootloader (BIOS + UEFI)	GPL 3
BusyBox	busybox.net	1.36	Utilidades POSIX mínimas	GPL 2
Python	python.org	3.12	Runtime de AgentD	PSF
Flask	palletsprojects.com	3.0	Framework web (GUI server)	BSD
python-socketio	python-socketio.readthedocs.io	5.10	WebSocket para GUI	BSD
httpx	github.com/encode/httpx	0.27	Cliente HTTP (API LLM)	BSD
Weston	wayland.freedesktop.org	13	Compositor Wayland	MIT/GPL
cog	github.com/igalia/cog	0.18	Navegador WPE Wayland	MIT
faster-whisper	github.com/SYSTRAN/faster-whisper	1.0	STT offline (modelo small ~140 MB)	MIT

Piper TTS	github.com/rhasspy/piper	1.0	TTS offline (modelo es ~50 MB)	MIT
Ollama Cloud	ollama.com	remoto	Inferencia LLM remota (alternativa a local)	Propietario
DeepSeek v4.1	models via Ollama	flash	Modelo LLM usado en desarrollo	MIT (código), datos propios

8.1 Requisitos de Compilación (VPS)

- GCC 12 y G++ 12 (Ubuntu 24.04: `sudo apt install gcc-12 g++-12`)
- Git, Mercurial, Python 3, libssl-dev, zlib1g-dev, libffi-dev, ncurses-dev
- ~50 GB de espacio libre para build/buildroot y build/output
- Conexión a internet (descargas de fuentes, compilación de 2-3 horas)

8.2 Requisitos de Ejecución (Portátil)

- **CPU:** x86-64 moderno (2010+), preferiblemente con AVX2 para mejor inferencia LLM
- **RAM:** 8 GB (repartido entre kernel, agente, modelo LLM local si se elige)
- **Almacenamiento:** 20 GB mínimo en disco/USB
- **GPU (opcional):** NVIDIA (CUDA) o AMD (ROCM) acelera STT y LLM local
- **Audio:** micrófono USB compatible con ALSA, altavoz o auriculares